

Verifying the Autoware Path Planner using Reachability Analysis During Adversarial Attacks

Thayer Hicks
Computer Science
UNC Chapel Hill
thayer@cs.unc.edu

Abstract—Autonomous vehicles require extensive safety analysis in order to avoid real-life catastrophes. Due to their heavy reliance on various sensor inputs, analysis on the effects of manipulations to these sensors provide additional safety reassurances. In this work, we present Auto-Reach, a toolbox that integrates simulation, sensor manipulation, trajectory extraction, and offline reachability analysis to further investigate the safety of the Autoware path planner. Using Autoware, AWSIM, and CommonRoad-Reach, we simulate attacks on LiDAR and GNSS inputs and analyze the resulting trajectories. Our approach focuses on modularity and reproducibility, enabling researchers to model various sensor manipulations and use a subset of the toolbox. Initial experiments show how simple attacks cause the planner to act more conservatively, causing the reachability analysis to identify fewer unsafe trajectories. Auto-Reach provides a practical toolbox for researchers and developers to test planning stacks against various threats.

I. INTRODUCTION

As autonomous vehicles become increasingly integrated into the daily lives of humans around the world, ensuring their safety is critical. Incomplete path planners, coupled with sensor malfunction, can lead to catastrophic outcomes. Reachability analysis, a form of model checking, is a useful tool when validating path planners for autonomous systems. However, creating tools that are able to provide this analysis for these systems introduces challenges around compute power, modeling accuracies, and complex setup. Additionally, developers must use simulations of the autonomous vehicle software stack because of physical and monetary limitations. Despite the rapidly growing use of open-source software like Autoware, there is a lack of accessible, easy to use tools that aid in the offline verification of path planners, especially in the face of adversarial sensor attacks.

We present Auto-Reach, a tool that connects Autoware, AWSIM, and CommonRoad-Reach to perform offline path validation. This is the first software suite that provides an easy way to record and validate the Autoware path planner. We introduce tools for map translation, set inclusion, and data collection, and we bridge the gap between theory and implementation for three widely used tools.

Beyond natural uncertainties, we investigate how adversarial attacks affect path safety. We provide a framework for defining and validating scenarios with an adversarial agent that has an unbounded ability to interfere with sensor data. This is the first attempt in the field to combine security analysis, autonomous vehicle caravan principles, and path validation.

II. STATE OF THE ART

A. CommonRoad-Reach and Zonotopes

Our work utilizes the CommonRoad-Reach toolbox, which leverages the power of zonotope computations [1]. Zonotopes are used to efficiently over-approximate the set of all possible states that an agent can reach under reasonable uncertainty [2]. The provided work introduces a theoretical framework which is capable of local linearization of a nonlinear system by utilizing a first-order Taylor expansion and its Lagrange remainder. This allows for the computation of all reachable sets represented by zonotopes. The power of this framework approach is that the computed set is a conservative over-approximation which allows all possible states to be considered. This computational power is critical when dealing with uncertainties that can arise in a system considering the safety of autonomous vehicles.

Through the use of the CommonRoad-Reach toolbox, we are able to build and integrate these techniques into an autonomous vehicle framework for online verification in a manner that allows for rapid updates of safety guarantees as parameters are changed. The change of parameters in this work is not solely due to traditional variations but also the malicious intent from an adversary on the vehicles sensor data. The attacker's scope, δ , will be increased until the set of all possible states of our experimental ego vehicle, intersects with the set of other object's states, leading to an inability to ensure safety.

B. Tube Based MPC vs. Our Approach

A similar set of research operating within the same field of work is Tube-based model predictive control [3][4][5]. These works engage in tactics that are also used within our approach, such as leveraging the computation power of zonotopes and computing reachable sets using reachability analysis. Although Tube-based model approaches aim to ensure safety, their end goals are different. Tube-based MPC focuses more on using the analysis of reachable sets to design a law that controls the extent of uncertainty in which a system can endure while still verifying safety. The parameters of uncertainty are all considered to be natural, and are controlled throughout the entire experiment. This control ensures that the produced path the vehicle follows is safe.

Alternatively, our approach is designed with verification in mind rather than direct path safety. Instead of designing

a controller, we allow uncertainties to not only be subject to natural causes, but also adversarial sensor manipulations. Reachability analysis is used to assess the safety of Autoware - the existing path planner - under these uncertainties. The attacker's scope δ will be increased to determine the limits of which δ can be before a produced path is no longer safe, meaning that the produced reachable sets are intersecting with unsafe regions. Essentially, tube-based MPC does not allow a produced path to be unsafe, while we monitor the limits of outside threats before a produced path is unsafe.

C. Reachability Analysis for Online Verification

Reachability analysis is used to compute all of the possible states the vehicle can reach, ensuring safety even under uncertainties such as sensor noise, model inaccuracies and environmental disturbances [6] [7]. These works focus on ensuring that all reachable sets are computed, while considering the stated uncertainties that might occur and ensuring there is no intersection with any unsafe states, guaranteeing safety. Our work extends the field of uncertainty to the next level by considering intentional malicious manipulation of sensor data by an adversary. We aim to explore the limits of the attacker's scope, δ , on our environment before safety cannot be ensured anymore. This is done by increasing the amount of havoc an adversary has, δ , before the Autoware path planner can no longer ensure safety.

III. BACKGROUND

Our proposal relies heavily on the CommonRoad-Reach toolbox, which specializes in reachability analysis within autonomous vehicles. Reachability encapsulates the set of all possible states a vehicle could reach within a given time, using this data in order to verify safety for a given path. In our case, a safe path is one that does not intersect with any adversaries and doesn't exit the reachable set. CommonRoad-Reach focuses on both polytopic set and graph based propagation. Polytopic set propagation aims to characterize the states an autonomous vehicle can reach as polytopes, demonstrating the possible set of states a given autonomous vehicle could feasibly attain within a given time. Likewise, graph based propagation constructs a graph in which nodes represent states and edges represent transitions, translating polytopic set propagation into a visual form that can be read and understood.

Alongside this, we will be defining our attacker in this scenario as an individual who can directly influence various sensors of the autonomous vehicle. This attacker's main goal is to influence the line of travel in a manner that would cause the vehicle and a potential obstacle to occupy the same point at the same time. This would be achieved by systematically altering the sensors until the path planner guides the vehicle to a point where, no matter what further instructions are inputted, the given vehicle would be unable to provide a valid path that ensures its safety.

As autonomous vehicle platoons and caravans become more common, evaluating their resilience to security breaches becomes critical. Vehicle-to-vehicle (V2V) communication al-

lows for autonomous vehicles to synchronize movements in order to maintain high precision. However, a false perception in a single vehicle could potentially lead to dangerous movements that following cars can blindly follow, because all sensors are falsely reporting normal conditions [8]. A study done by researchers in the platooning simulator PLEXE, demonstrated that in a platoon of autonomous vehicles where all vehicles were traveling 100 km/h, if the lead vehicle is forced to a sudden halt, the following vehicles are unable to brake in time leading to collisions [9]. The study also notes that the simulation is an example of a local attack, and if scaled to a nationwide level, with a mixture of non-autonomous vehicles, the resulting damage would be significantly greater.

IV. THREAT MODEL

As autonomous vehicle platoons and caravans become more common, evaluating their resilience to security breaches is critical. Autoware correctly implements emergency response systems, chief among these is the vehicle's emergency stop procedure. We assume an actor whose goal is to cause the vehicle to crash, without triggering emergency halt procedure. The actor has unlimited control of the vehicle's sensor data streams, and can manipulate them in a number of ways. However, the actor can not directly interfere with the vehicle actuation or override safety-critical modules. The actor can operate on ROS topics, and through those feed manipulated data into the path planner.

V. APPROACH

Our approach consists of four major pieces: defining and running scenarios using AWSIM and Autoware, injecting manipulated sensor data on ROS topics, recording trajectory data via ROS2 bags, and verifying path safety using offline reachability analysis with CommonRoad-Reach.

A. Setup

In order to accurately simulate real-world scenarios under adversarial conditions, we use AWSIM and Autoware. We have chosen to use offline verification to verify the planning module after the simulation has completed. We forgo the use of a physical vehicle and use the open source simulator AWSIM, along with Autoware as the autonomous software stack. AWSIM provides a near photorealistic environment with the added benefit of pre-built environments and relative ease of use. The sensors that will be manipulated in the experiment will be the default sensors on the EgoVehicle in AWSIM, namely the Pose and LiDAR. To run the computationally expensive simulations, we use a machine with NVIDIA RTX 4080 Super GPUs that has been proven in the past to perform well with these simulations. Because the verification will occur after the simulation has ended, we can expect normal, high quality simulations.

B. Verification Approach

To determine the validity of the Autoware planner, we will use Reachability Analysis, leaning heavily on the toolbox

CommonRoad-Reach. First, a scenario will be designed with obstacle vehicles and pedestrians. Second, the simulation will be initialized, with a traceable ego-vehicle location and goal pose. Third, the simulation will run while a rosbag is recorded, which will keep track of information like vehicle pose, obstacle poses, and trajectory points. Finally, we will convert this data, using a static map, and place all of the objects onto the map suitable for CommonRoad-Reach to verify the trajectories of the car throughout timestamps in the simulation.

C. Attack Scenarios

One benefit of relying on a simulation-based experiment is that we are able to easily perform sensor attacks automatically. Because of the underlying pub-sub nature of Autoware, which uses Robot Operating System (ROS) 2, we perform the following:

- Interleave falsified sensor data on existing ROS topics (LiDAR, Pose).

And, in future work have the ability to perform additional disturbances, such as:

- Intercepting ROS topics to manipulate them before sending them to Autoware.
- Sensor silencing, either altogether or for varying periods of time.

With these attack scenarios, we will be able to influence the planned trajectories of the simulation vehicles in ways that will eventually break their guarantee of crash avoidance.

D. Evaluation Methods

Our experiments will start by evaluating a control scenario, using CommonRoad as an offline verification system. The standard Autoware planner should serve as a baseline of verification from which the effects of our attacker's attacks will be measured. Then, we will give the attacker increased control over the data sent to Autoware by the car's sensors and aim to repeat the experiment. Using a simulated model of the car gives us some benefits of reproducibility because we can define common starting and goal positions. However, due to the nondeterministic nature of the simulated environment and the Autoware planner, the evaluation of our experiments provide preliminary but non-exhaustive validation.

VI. IMPLEMENTATION

To verify the Autoware planner, a large amount of setup and data manipulation was required. First, the CommonRoad-Reach supplies minimal documentation, and doesn't provide a robust method of translating between Autoware and the toolbox, especially when coupled with AWSIM. Nonetheless, a translation of maps, coordinates, and obstacle initializations is possible with a large amount of setup. Second, due to AWSIM's change to use prebuilt LiDAR packages, intercepting and altering data is difficult. To overcome these challenges, we have created a flexible and thorough set of tools to connect AWSIM, Autoware, and CommonRoad-Reach.

A. Autoware with AWSIM

Due to physical constraints, autonomous vehicle simulators are widely used in the industry. One of the leaders in the open source simulator realm is AWSIM, which is designed to integrate with Autoware. To create a working environment, the user is given a lofty set of hardware requirements, namely an NVIDIA RTX2080Ti GPU. While this setup is capable of running the software, we recommend at least an RTX4080Ti to maintain an acceptable frame rate while performing computationally expensive operations, like collecting a rosbag. Second, the user is tasked with building the software suite. With robust documentation and the use of ansible scripts, the suite has simple setup instructions. However, this simplicity introduces added difficulty in error-handling. For this paper, the software was built from source and we found that these versions were optimal for reducing errors: NVIDIA GPU Driver version 550.X, Autoware release 0.43.1, AWSIM release 1.3.1, Unity Editor version 2021.1.7f1, and Python version 3.10.X.

B. Adding Obstacles in AWSIM

Because AWSIM is open source, it allows for easy manipulation to create custom scenarios. Here, we add three static obstacles: two pedestrians that are each in a crosswalk, and one parked van placed after the second crosswalk. This is done by opening AWSIM in the Unity editor, dragging the appropriate prefab into the scene, and performing adequate testing to find the optimal positioning of the objects. Careful manual tuning was necessary in this process. We found that if an object was too much in the way of the path, the car would stop indefinitely, and, if the object was too far out of the car's path, it wouldn't meaningfully impact the simulation. Realistic static objects help us to verify a more lifelike scenario, something that is lacking in the CommonRoad-Reach example files.

Next, because there isn't a uniform coordinate system between Autoware and AWSIM, we implemented a translation. AWSIM uses a Unity world coordinate system, with axes oriented as:

- X-axis: Right
- Y-axis: Up
- Z-axis: Forward

However, Autoware employs a typical ROS ENU (East-North-Up) coordinate frame (see Fig. 1). As a result, a rotation and reordering of axes is required to align the two systems. We manually derived the following transformation to map Autoware coordinates to AWSIM (Unity) coordinates: $(x_{AU}, y_{AU}, z_{AU}) \rightarrow (-y_{AW}, z_{AW}, x_{AW})$. Additionally, there is a fixed offset between the two coordinate frames. Because significant documentation regarding coordinate transformations was lacking, we manually derived the mapping between AWSIM and Autoware. To do this, we first initialized a simulation and subscribed to the `/localization/pose_estimator/pose` ROS topic. With these coordinates of the stationary *EgoVehicle*, we were then able to deduce a finite translation of coordinates by comparing the Autoware pose to the listed *EgoVehicle*

position in the AWSIM Unity project. Through this method we found that the translation is as follows: $(x_{AU}, y_{AU}, z_{AU}) - (81655.73375, 49696.3545312, 42.470413818) \rightarrow (-y_{AW}, z_{AW}, x_{AW})$. Correct alignment of coordinate frames was essential to map our static obstacles accurately across AWSIM and Autoware environments.

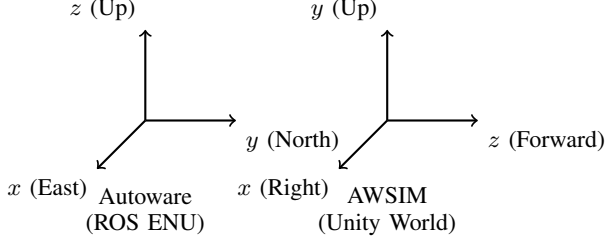


Fig. 1. Coordinate frames used by Autoware and AWSIM.

C. Lanelet2 to CommonRoad

Because CommonRoad uses a custom XML-based file format, translations of standard Autoware maps (Lanelet2) map files are essential for enabling reachability analysis. Fortunately, CommonRoad provides an interface for these translations. Using the *crdesigner.map_conversion* Python package, we were able to translate the standard Autoware map to the CommonRoad XML format, with two caveats:

- The traffic lights in the standard Autoware map were translated to have an (x, y) value of (inf, inf) . This caused issues in the reachability analysis, so we removed all traffic lights from the CommonRoad map. The reachability analysis tool doesn't rely on traffic lights, so this did not cause any other issues.
- The translation, which used the proper UTM Zone of 54, did not translate the coordinates from the Autoware map correctly, so an additional manual translation was required. To do this, we queried the minimum (x, y) values in both the Autoware and CommonRoad map files to find the necessary translation. Additionally, we queried the maximum (x, y) values and found an insignificant difference in the required translation, proving that there were no scaling inconsistencies.

In response to these translation inconsistencies, we devised a tool to manually correct faulty CommonRoad map files to ensure map integrity. By cross-validating the coordinate bounds, we verified that our translation maintained translation fidelity.

D. Modeling Obstacles in CommonRoad

To properly model the system for reachability analysis, we added obstacles that were implemented in AWSIM into the CommonRoad map file. Using the previously established coordinate translation and corrected map file, together with the *commonroad.scenario* Python package, we were able to model the pedestrians as circles and the van as a rectangle. By modeling these obstacles as over-approximations, we ensure

conservative safety bounds during reachability analysis, which minimizes the risk of underestimating potential collisions. Because of the absence of direct integration between AWSIM and CommonRoad map definitions, we developed a workflow for obstacle modeling.

E. Publishing Faulty Sensor Data

To model adversarial manipulation of crucial inputs to the Autoware planning pipeline, we interleave falsified data. We devise a scenario where the attacker's goal is to alter sensor data enough to jeopardize the trajectory while avoiding the situation of the car stopping altogether. One method to do this is to interleave faulty LiDAR data with the correct data coming from the AWSIM simulation. To do this, we first recorded an unaltered run of the simulation into a rosbag, only recording data from the `/sensing/lidar/concatenated/pointcloud` topic. Next, we de-serialized the first instance of that data and stored it for use as our faulty data. Then, we created a new publisher to the `/sensing/lidar/concatenated/pointcloud` topic that would send messages at a configurable rate. This maintained message validity and avoided immediate fault detection. Through extensive testing, it was found that a publishing rate of around 1.0 Hz successfully disrupted the planning module while also avoiding a complete shutdown. The result of our work was a normal-looking stream of LiDAR data with one message of faulty data interleaved every second. Additionally, we did the same process with the `/sensing/lidar/concatenated/pointcloudtopic`, interleaving faulty pose messages. Through extensive testing, we found that 0.4 Hz was an optimal publishing rate. When published at a rate of 0.5 Hz or higher, the path planner would instantly enter an emergency protocol.

F. Simulation

To ensure reproducibility and accurate offline verification, we performed controlled simulations using AWSIM. Using the default initial pose of the *EgoVehicle*, we ran a control simulation with no sensor manipulation, and simulations with manipulations. During the simulations, we recorded messages published on the `/planning/scenario_planning/trajectory` topic into a rosbag. Next, for each trajectory published, we de-serialized the data and created a list of trajectory points, using the first one as the current *EgoVehicle* position. Then, we checked each point for intersection in the reachable set using a bounded box check. If a point was found to lie outside of the reachable set, we marked the trajectory as faulty and looked for more points inside the same trajectory that fell outside of the reachable set.

G. Verification using CommonRoad-Reach

After collecting a large amount of trajectory data, we used the CommonRoad-Reach toolbox to calculate reachable sets, and then used our bounded box method to validate path safety. First, we needed to build the configuration based on the CommonRoad map file. Next, we used the

ReachableSetInterface class to compute the reachable step for each *EgoVehicle* position in our simulation. We found that the CommonRoad-Reach toolbox was designed with visualization as a higher priority than numerical validation, so we manually checked each point within each polygon in the reachable set. Our workflow used the Python backend in the reachable set tool, which was magnitudes less efficient than the alternative C++ backend but exposed an interface to use specific boundaries within the reachable set. For example, processing 250 trajectories with the Python backend took around 6 hours, and recordings ranged from 250 to 500 trajectories. Because each polygon that was generated by the reachable set tool was an axis-aligned rectangle, we were able to perform a concise check to reason whether a given point was inside a subset of the reachable set. By checking all polygons in the set, we were able to determine if a given point was inside the reachable set (see Fig. 2).

```

for point in path:
    for node in nodes:
        if point.x <= node.p_x_max
           and point.x >= node.p_x_min
           and point.y <= node.p_y_max
           and point.y >= node.p_y_min:
            inside = True
            break

```

Fig. 2. Checking if a trajectory point lies inside the reachable set polygons (simplified).

VII. EVALUATION

A. Experiment Setup

We conducted three simulations of autonomous driving with Autoware and AWSIM. In all three, the map, starting pose, goal pose, and stationary obstacles were identical. The first simulation had no sensor interference, the second had LiDAR interference at a frequency of 1.0 Hz, and the third had pose interference at a frequency of 0.4 Hz. In all three cases, the reachable set was generated for each position of the car, and all points on each trajectory were evaluated for intersection with the reachable set.

B. Results

In the scenario with no interference, 259 trajectories were evaluated, and 196 of those had at least one point that fell outside of the computed reachable set. Each trajectory contained 168 points, so out of a possible 43,512 points, 2824 fell outside of the reachable set.

In the scenario with LiDAR interference, 446 trajectories were evaluated, and 295 of those had at least one point that fell outside of the computed reachable set. Each trajectory contained 168 points, so out of a possible 74,928 points, 1317 fell outside of the reachable set.

In the scenario with Pose interference, 239 trajectories were evaluated, and 173 of those had at least one point that fell outside of the computed reachable set. Each trajectory

contained 168 points, so out of a possible 40,152 points, 2569 fell outside of the reachable set.

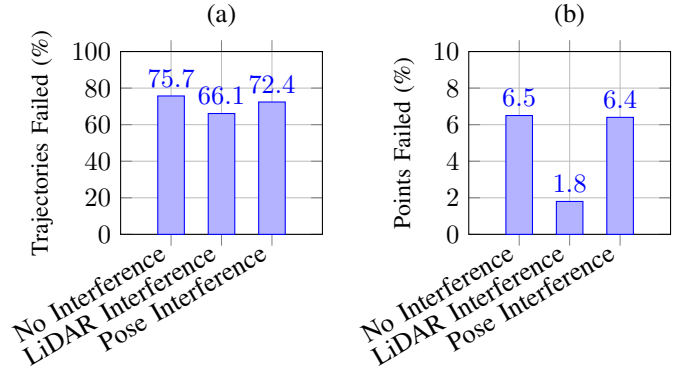


Fig. 3. Percentage of trajectory failures (a) and point failures (b) across three scenarios: no interference, LiDAR interference, and pose interference.

C. Analysis

We found a difference in the validity of paths generated under the three scenarios. First, there was a noticeable difference in the validity of the LiDAR Interference scenario when compared to the No Interference scenario. Unexpectedly, the path planner under LiDAR interference performed better than normal. We suspect that the LiDAR attack caused the planner to behave more conservatively, which benefited its path validation statistics. This could also be seen on the simulation, as the car traveled much more slowly and applied sudden brake force at regular intervals.

Similarly interesting was how the Pose Interference scenario performed similarly to the No Interference scenario. During the simulation, the car appeared to act more like that in the LiDAR Interference simulation. We suspect that the reduced frequency of interleaved data could be the cause of this.

With no interference, an alarmingly high failure rate was observed (75%). We speculate that this is due to sensor and planning noise, conservatism in the reachability analysis tool, or other shortcomings. This exemplifies the need for future coupling of the planning and validation process.

D. Limitations

The first limitation we experienced was the sheer amount of time required to perform adequate tests. Because we had to use the python backend, evaluating one simulation took anywhere between six and twelve hours. Because of this, our number of validated scenarios is low. While more data would have provided some insight into the types of actions that caused the planner to fail, we have provided an intuitive framework for future testing.

Second, the CommonRoad-Reach tool left a lot to be desired. First, we were expecting the tool to have a more robust method of validation. However, the tool turned out to be focused on the visualization of the reachable set. While it was useful to see these sets visualized, we would have benefited from more attention to validation from the developers.

Additionally, we found that the reachable sets generated in the example scenarios were clean and covered the area reasonably well with large polygons. However, when running the tool on more production-level map data, we found the polygons to be small and often left some areas that appeared to be reachable unaccounted for.

VIII. CONCLUSION

In this paper we introduce Auto-Reach, a framework for conducting offline verification of the Autoware path planner. Using our framework, we evaluated the Autoware path planner under three attack scenarios: no interference, some LiDAR interference, and some pose interference. Our verification showed that around seventy percent of trajectories sampled contained at least one point that fell outside of the reachable set.

Our work helped close the gap between theory and implementation of autonomous software tools. We pave the way for future work surrounding the verification of path planners, that play a crucial role in the safety of autonomous vehicles. In the future, we expect to gather more data, establish more scenarios, and use our tools in conjunction with path planner development.

REFERENCES

- [1] E. I. Liu, G. Würsching, M. Klischat and M. Althoff, "CommonRoad-Reach: A Toolbox for Reachability Analysis of Automated Vehicles," 2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC), Macau, China, 2022, pp. 2313-2320, doi: 10.1109/ITSC55140.2022.9922232. keywords: Heuristic algorithms;Dynamics;Benchmark testing;Real-time systems;Planning;Vehicle dynamics;Reachability analysis
- [2] M. Althoff, O. Stursberg and M. Buss, "Reachability analysis of nonlinear systems with uncertain parameters using conservative linearization," 2008 47th IEEE Conference on Decision and Control, Cancun, Mexico, 2008, pp. 4042-4048, doi: 10.1109/CDC.2008.4738704. keywords: Reachability analysis;Nonlinear systems;Safety;Linear systems;Nonlinear dynamical systems;State-space methods;Nonlinear control systems;Control systems;Continuous time systems;Automatic control
- [3] Gonzalez, Richard & Fiacchini, Mirko & Alamo, Teodoro & Guzmán, José & Rodríguez, Francisco. (2011). Online Robust Tube-based MPC for Time-Varying Systems : A Practical Approach. *International Journal of Control*. 84. 1157-1170. 10.1080/00207179.2011.594093.
- [4] S. V. Rakovic, B. Kouvaritakis, M. Cannon, C. Panos and R. Findeisen, "Parameterized Tube Model Predictive Control," in *IEEE Transactions on Automatic Control*, vol. 57, no. 11, pp. 2746-2761, Nov. 2012, doi: 10.1109/TAC.2012.2191174. keywords: Electron tubes;Robustness;Proposals;Predictive control;Optimal control;Optimization;Linearity;Constrained control;convex programming;robust control;set-dynamics;tube model predictive control
- [5] J.M. Bravo, T. Alamo, E.F. Camacho, Robust MPC of constrained discrete-time nonlinear systems based on approximated reachable sets, *Automatica*, Volume 42, Issue 10, 2006, Pages 1745-1751, ISSN 0005-1098, <https://doi.org/10.1016/j.automatica.2006.05.003>.
- [6] M. Althoff and J. M. Dolan, "Online Verification of Automated Road Vehicles Using Reachability Analysis," in *IEEE Transactions on Robotics*, vol. 30, no. 4, pp. 903-918, Aug. 2014, doi: 10.1109/TRO.2014.2312453. keywords: Vehicles;Trajectory;Mathematical model;Vehicle dynamics;Reachability analysis;Safety;Roads;Automated vehicles;autonomous cars;formal verification;reachability analysis;set-based computation;Automated vehicles;autonomous cars;formal verification;reachability analysis;set-based computation
- [7] M. Althoff and J. M. Dolan, "Set-based computation of vehicle behaviors for the online verification of autonomous vehicles," 2011 14th International IEEE Conference on Intelligent Transportation Systems (ITSC), Washington, DC, USA, 2011, pp. 1162-1167, doi: 10.1109/ITSC.2011.6083052. keywords: Vehicles;Trajectory;Vehicle dynamics;Noise;Roads;Reachability analysis;Computational modeling
- [8] A. Shaikh, M. U. Khan, N. Javaid, A. A. Shah, and F. Al-balooshi, "Vehicular Platoon Communication: Architecture, Security Threats and Open Challenges," *Vehicles*, vol. 3, no. 3, pp. 360-385, Sep. 2021, doi: 10.3390/vehicles3030025. keywords: Vehicles;Vehicle dynamics;Vehicle-to-vehicle,
- [9] J. Cheon, D. Jang, Y. Lee, and J. Jeon, "Secure and safe automated vehicle platooning," *IEEE/AES Systems Magazine*, vol. 31, no. 8, pp. 28-35, Aug. 2016, doi: 10.1109/MAES.2016.150165. keywords: Vehicles;Trajectory;Vehicle dynamics;PLEXE

ACKNOWLEDGMENT

The authors acknowledge the use of AI tools (ChatGPT 4.0) for assistance in editing LaTeX code, reasoning about organization, grammar checking, and formatting small code snippets. All technical content, experimental design, and analysis were conducted by the authors.